Helicopter Game - Full Chat Transcript

Pygame helicopter lab, Tasks 1 to 4

User (message 1)

I'm working on a Pygame helicopter game. I'll paste my files one at a time. Please don't change anything yet, just read them.

Claude (reply 1)

Sounds good. Go ahead and paste your first file whenever you're ready. I'll read through each one without making any changes or suggestions until you tell me what you'd like done.

If it helps, you can include the filename above each paste so I can keep track of how the pieces fit together.

User (message 2)

[Pasted five files]

MAIN.PY

```
"""
Helicopter Game (Lab Starter)

Run with:  python3 main.py

Controls: Up/Down arrows to move.
"""

import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE


def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Helicopter")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 22)

    engine = GameEngine()
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN:
                engine.handle_keydown(event.key)

        keys = pygame.key.get_pressed()
        engine.handle_input(keys)
        engine.update()
        engine.draw(screen, font)

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()


if __name__ == "__main__":
    main()
```

HELICOPTER.PY

```
"""
Helicopter: the player-controlled vehicle. Moves vertically based on
held Up/Down keys.
"""

import pygame

THRUST = 0.4


class Helicopter:
    def __init__(self, x, y, width=40, height=24):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.vy = 0.0

    def handle_input(self, keys_pressed):
        if keys_pressed[pygame.K_UP]:
            self.vy -= THRUST
        if keys_pressed[pygame.K_DOWN]:
            self.vy += THRUST

    def update(self, height_bound):
        self.y += self.vy
        if self.y < 0:
            self.y = 0
            self.vy = 0
        # NOTE: no corresponding check against the bottom boundary

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.width / 2), int(self.y - self.height / 2),
            self.width, self.height,
        )
```

GAME_ENGINE.PY

```
"""
GameEngine: owns the helicopter and all obstacles.

Starter version: the helicopter moves and obstacles scroll by, but
there's no collision detection at all yet (the helicopter can fly
straight through obstacles harmlessly), no scoring, and no shield.
That's Tasks 2, 3, and 4. Movement also has known bugs (see
game/helicopter.py) that Task 1 asks you to fix.
"""

import random

from game.helicopter import Helicopter
from game.obstacle import Obstacle
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 90
GAP_HEIGHT = 150
WALL_WIDTH = 60
SCROLL_SPEED = 3


class GameEngine:
    def __init__(self):
        self.helicopter = Helicopter(x=100, y=HEIGHT / 2)
        self.obstacles = []
        self.frames_until_spawn = 0

    def _spawn_obstacle(self):
        margin = 60
        gap_y = random.randint(margin + GAP_HEIGHT // 2, HEIGHT - margin - GAP_HEIGHT // 2)
        self.obstacles.append(Obstacle(
            x=WIDTH, gap_y=gap_y, gap_height=GAP_HEIGHT,
            wall_width=WALL_WIDTH, screen_height=HEIGHT, speed=SCROLL_SPEED,
```

```
        ))

    def handle_input(self, keys_pressed):
        self.helicopter.handle_input(keys_pressed)

    def handle_keydown(self, key):
        pass

    def update(self):
        self.helicopter.update(HEIGHT)

        self.frames_until_spawn -= 1
        if self.frames_until_spawn <= 0:
            self._spawn_obstacle()
            self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

        for obstacle in self.obstacles:
            obstacle.update()
        self.obstacles = [o for o in self.obstacles if not o.is_off_screen()]

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.helicopter, self.obstacles)
```

OBSTACLE.PY

```
"""
Obstacle: a scrolling wall pair with a gap the helicopter must fly
through.
"""

import pygame


class Obstacle:
    def __init__(self, x, gap_y, gap_height, wall_width, screen_height, speed):
        self.x = x
        self.gap_y = gap_y
        self.gap_height = gap_height
        self.wall_width = wall_width
        self.screen_height = screen_height
        self.speed = speed
        self.scored = False   # used for distance/pass tracking later

    def update(self):
        self.x -= self.speed

    def is_off_screen(self):
        return self.x + self.wall_width < 0

    def get_top_rect(self):
        top_height = self.gap_y - self.gap_height / 2
        return pygame.Rect(int(self.x), 0, self.wall_width, int(top_height))

    def get_bottom_rect(self):
        bottom_y = self.gap_y + self.gap_height / 2
        return pygame.Rect(int(self.x), int(bottom_y), self.wall_width, int(self.screen_height - bottom_y))
```

RENDERER.PY

```
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (140, 200, 230)
COLOR_HELI = (60, 60, 70)
COLOR_OBSTACLE = (70, 150, 80)
COLOR_TEXT = (20, 20, 20)

def draw_scene(surface, helicopter, obstacles):
```

```
    surface.fill(COLOR_BG)
    for obstacle in obstacles:
        pygame.draw.rect(surface, COLOR_OBSTACLE, obstacle.get_top_rect())
        pygame.draw.rect(surface, COLOR_OBSTACLE, obstacle.get_bottom_rect())
    pygame.draw.rect(surface, COLOR_HELI, helicopter.get_rect(), border_radius=4)


def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)


def draw_banner(surface, font, text):
    surf = font.render(text, True, (180, 40, 40))
    rect = surf.get_rect(center=(surface.get_width() // 2, surface.get_height() // 2))
    surface.blit(surf, rect)
```

In `Helicopter.handle_input`, vertical speed `vy` keeps growing while a key is held, so reversing direction feels laggy. Cap `vy` to a maximum (like ±8) so it can't build up forever. Also, `Helicopter.update` only checks the top edge (`y < 0`). Add the matching bottom check using the screen height and helicopter height so it can never leave the screen. Give me the complete updated `helicopter.py` file.

Claude (reply 2)

Here's the updated `helicopter.py`:

helicopter.py

```
"""
Helicopter: the player-controlled vehicle. Moves vertically based on
held Up/Down keys.
"""

import pygame

THRUST = 0.4
MAX_SPEED = 8


class Helicopter:
    def __init__(self, x, y, width=40, height=24):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.vy = 0.0

    def handle_input(self, keys_pressed):
        if keys_pressed[pygame.K_UP]:
            self.vy -= THRUST
        if keys_pressed[pygame.K_DOWN]:
            self.vy += THRUST
        # Cap speed so it can't build up forever
        self.vy = max(-MAX_SPEED, min(MAX_SPEED, self.vy))

    def update(self, height_bound):
        self.y += self.vy

        # self.y is the helicopter's center, so keep the whole body on screen
        half_height = self.height / 2
        if self.y - half_height < 0:
            self.y = half_height
            self.vy = 0
        elif self.y + half_height > height_bound:
            self.y = height_bound - half_height
            self.vy = 0

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.width / 2), int(self.y - self.height / 2),
            self.width, self.height,
        )
```

Changes:

- **Speed cap:** I added a `MAX_SPEED = 8` constant, and `vy` is clamped to `±MAX_SPEED` at the end of `handle_input`.
- **Bottom check:** `update` now stops the helicopter at `height_bound` and zeroes `vy`, mirroring the top.
- **Top check also changed:** since `y` is the helicopter's center (see `get_rect`), the original `y < 0` check let the top half of the sprite poke off the screen. I changed it to use `half_height` as well so both edges behave the same. If you'd rather keep the original top check untouched, change that line back to `if self.y < 0:` with `self.y = 0`.

User (message 3)

[Pasted the files again, with blank lines removed, plus the updated helicopter.py]

MAIN.PY

```
"""
Helicopter Game (Lab Starter)
Run with:  python3 main.py
Controls: Up/Down arrows to move.
"""
import pygame
from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE
def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Helicopter")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 22)
    engine = GameEngine()
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN:
                engine.handle_keydown(event.key)
        keys = pygame.key.get_pressed()
        engine.handle_input(keys)
        engine.update()
        engine.draw(screen, font)
        pygame.display.flip()
        clock.tick(60)
    pygame.quit()
if __name__ == "__main__":
    main()
```

OBSTACLE.PY

```
"""
Obstacle: a scrolling wall pair with a gap the helicopter must fly
through.
"""
import pygame
class Obstacle:
    def __init__(self, x, gap_y, gap_height, wall_width, screen_height, speed):
        self.x = x
        self.gap_y = gap_y
        self.gap_height = gap_height
        self.wall_width = wall_width
        self.screen_height = screen_height
        self.speed = speed
        self.scored = False   # used for distance/pass tracking later
    def update(self):
        self.x -= self.speed
    def is_off_screen(self):
        return self.x + self.wall_width < 0
    def get_top_rect(self):
```

```
        top_height = self.gap_y - self.gap_height / 2
        return pygame.Rect(int(self.x), 0, self.wall_width, int(top_height))
    def get_bottom_rect(self):
        bottom_y = self.gap_y + self.gap_height / 2
        return pygame.Rect(int(self.x), int(bottom_y), self.wall_width, int(self.screen_height - bottom_y))
```

RENDERER.PY

```
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""
import pygame
WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)
COLOR_BG = (140, 200, 230)
COLOR_HELI = (60, 60, 70)
COLOR_OBSTACLE = (70, 150, 80)
COLOR_TEXT = (20, 20, 20)
def draw_scene(surface, helicopter, obstacles):
    surface.fill(COLOR_BG)
    for obstacle in obstacles:
        pygame.draw.rect(surface, COLOR_OBSTACLE, obstacle.get_top_rect())
        pygame.draw.rect(surface, COLOR_OBSTACLE, obstacle.get_bottom_rect())
    pygame.draw.rect(surface, COLOR_HELI, helicopter.get_rect(), border_radius=4)
def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)
def draw_banner(surface, font, text):
    surf = font.render(text, True, (180, 40, 40))
    rect = surf.get_rect(center=(surface.get_width() // 2, surface.get_height() // 2))
    surface.blit(surf, rect)
```

HELICOPTER.PY

```
"""
Helicopter: the player-controlled vehicle. Moves vertically based on
held Up/Down keys.
"""

import pygame

THRUST = 0.4
MAX_SPEED = 8


class Helicopter:
    def __init__(self, x, y, width=40, height=24):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.vy = 0.0

    def handle_input(self, keys_pressed):
        if keys_pressed[pygame.K_UP]:
            self.vy -= THRUST
        if keys_pressed[pygame.K_DOWN]:
            self.vy += THRUST
        # Cap speed so it can't build up forever
        self.vy = max(-MAX_SPEED, min(MAX_SPEED, self.vy))

    def update(self, height_bound):
        self.y += self.vy

        # self.y is the helicopter's center, so keep the whole body on screen
        half_height = self.height / 2
        if self.y - half_height < 0:
            self.y = half_height
            self.vy = 0
        elif self.y + half_height > height_bound:
            self.y = height_bound - half_height
            self.vy = 0

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.width / 2), int(self.y - self.height / 2),
```

```
            self.width, self.height,
        )
```

Now Task 2. Add collision detection between the helicopter and the obstacles. Touching either the top wall or the bottom wall of an obstacle should end the game and display a clear "GAME OVER" message on screen. Flying through the gap must never end the game, no matter where in the gap the helicopter is. Use rect-based collision, checking the top wall and bottom wall rects separately. Also let me restart with the R key after game over. Tell me which files changed, and give me the complete updated version of each one.

Claude (reply 3)

Only one file changed: `game/game_engine.py`. `main.py` already forwards key presses to `engine.handle_keydown`, and `renderer.py` already has `draw_banner`, so neither needed edits. `helicopter.py` and `obstacle.py` are also unchanged.

game/game_engine.py

```
"""
GameEngine: owns the helicopter and all obstacles.

Starter version: the helicopter moves and obstacles scroll by, but
there's no scoring and no shield yet. That's Tasks 3 and 4.
Collision detection and game over / restart are handled here (Task 2).
"""

import random

import pygame

from game.helicopter import Helicopter
from game.obstacle import Obstacle
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 90
GAP_HEIGHT = 150
WALL_WIDTH = 60
SCROLL_SPEED = 3


class GameEngine:
    def __init__(self):
        self._reset()

    def _reset(self):
        """Put the game back into its starting state."""
        self.helicopter = Helicopter(x=100, y=HEIGHT / 2)
        self.obstacles = []
        self.frames_until_spawn = 0
        self.game_over = False

    def _spawn_obstacle(self):
        margin = 60
        gap_y = random.randint(margin + GAP_HEIGHT // 2, HEIGHT - margin - GAP_HEIGHT // 2)
        self.obstacles.append(Obstacle(
            x=WIDTH, gap_y=gap_y, gap_height=GAP_HEIGHT,
            wall_width=WALL_WIDTH, screen_height=HEIGHT, speed=SCROLL_SPEED,
        ))

    def _check_collisions(self):
        """Return True if the helicopter touches the top or bottom wall of any obstacle."""
        heli_rect = self.helicopter.get_rect()
        for obstacle in self.obstacles:
            # Check the two wall rects separately; the gap between them
            # is not part of either rect, so flying through it is safe.
            if heli_rect.colliderect(obstacle.get_top_rect()):
                return True
            if heli_rect.colliderect(obstacle.get_bottom_rect()):
                return True
        return False
```

```
    def handle_input(self, keys_pressed):
        if self.game_over:
            return
        self.helicopter.handle_input(keys_pressed)

    def handle_keydown(self, key):
        if self.game_over and key == pygame.K_r:
            self._reset()

    def update(self):
        if self.game_over:
            return

        self.helicopter.update(HEIGHT)

        self.frames_until_spawn -= 1
        if self.frames_until_spawn <= 0:
            self._spawn_obstacle()
            self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

        for obstacle in self.obstacles:
            obstacle.update()
        self.obstacles = [o for o in self.obstacles if not o.is_off_screen()]

        if self._check_collisions():
            self.game_over = True

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.helicopter, self.obstacles)
        if self.game_over:
            renderer.draw_banner(surface, font, "GAME OVER - Press R to restart")
```

What changed:

- **Collision:** `_check_collisions` uses `colliderect` on the helicopter rect against each obstacle's top and bottom wall rects separately. The gap isn't part of either rect, so any position inside it is safe.
- **Game over:** `update` runs the collision check after everything has moved, and sets `self.game_over = True` on a hit. While game over is set, `update` and `handle_input` do nothing, so the scene freezes in place under the banner.
- **Banner:** `draw` still draws the scene, then overlays "GAME OVER - Press R to restart" in the center using your existing `draw_banner`.
- **Restart:** `handle_keydown` resets the game when R is pressed during game over. I moved the starting state into a `_reset()` method that `__init__` also calls, so the initial state and the restart state can't drift apart. R does nothing during normal play.

User (message 4)

Now Task 3. Add a distance-based score. It should track how far the helicopter has traveled and increase automatically while the game is running (and stop increasing once the game is over). Show the current distance on screen during play, show the final distance on the game-over screen, and reset it to zero when a new game starts with R. Tell me which files changed, and give me the complete updated version of each one.

Claude (reply 4)

Only one file changed: `game/game_engine.py`. The distance counter lives in the engine, and `renderer.py` already has `draw_text`, which is all the HUD needs. `main.py`, `helicopter.py`, `obstacle.py`, and `renderer.py` are unchanged.

game/game_engine.py

```
"""
```

```
GameEngine: owns the helicopter and all obstacles.

Starter version: the helicopter moves and obstacles scroll by, but
there's no shield yet. That's Task 4.
Collision detection and game over / restart are handled here (Task 2),
along with the distance score (Task 3).
"""

import random

import pygame

from game.helicopter import Helicopter
from game.obstacle import Obstacle
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 90
GAP_HEIGHT = 150
WALL_WIDTH = 60
SCROLL_SPEED = 3
PIXELS_PER_METER = 10   # scroll pixels that count as one meter of distance


class GameEngine:
    def __init__(self):
        self._reset()

    def _reset(self):
        """Put the game back into its starting state."""
        self.helicopter = Helicopter(x=100, y=HEIGHT / 2)
        self.obstacles = []
        self.frames_until_spawn = 0
        self.game_over = False
        self.distance = 0.0   # in scroll pixels; converted to meters for display

    def _distance_meters(self):
        return int(self.distance / PIXELS_PER_METER)

    def _spawn_obstacle(self):
        margin = 60
        gap_y = random.randint(margin + GAP_HEIGHT // 2, HEIGHT - margin - GAP_HEIGHT // 2)
        self.obstacles.append(Obstacle(
            x=WIDTH, gap_y=gap_y, gap_height=GAP_HEIGHT,
            wall_width=WALL_WIDTH, screen_height=HEIGHT, speed=SCROLL_SPEED,
        ))

    def _check_collisions(self):
        """Return True if the helicopter touches the top or bottom wall of any obstacle."""
        heli_rect = self.helicopter.get_rect()
        for obstacle in self.obstacles:
            # Check the two wall rects separately; the gap between them
            # is not part of either rect, so flying through it is safe.
            if heli_rect.colliderect(obstacle.get_top_rect()):
                return True
            if heli_rect.colliderect(obstacle.get_bottom_rect()):
                return True
        return False

    def handle_input(self, keys_pressed):
        if self.game_over:
            return
        self.helicopter.handle_input(keys_pressed)

    def handle_keydown(self, key):
        if self.game_over and key == pygame.K_r:
            self._reset()

    def update(self):
        if self.game_over:
            return

        self.helicopter.update(HEIGHT)

        self.frames_until_spawn -= 1
```

```
        if self.frames_until_spawn <= 0:
            self._spawn_obstacle()
            self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

        for obstacle in self.obstacles:
            obstacle.update()
        self.obstacles = [o for o in self.obstacles if not o.is_off_screen()]

        if self._check_collisions():
            self.game_over = True
        else:
            # Only count distance for frames the helicopter survives
            self.distance += SCROLL_SPEED

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.helicopter, self.obstacles)

        distance_text = f"Distance: {self._distance_meters()} m"
        if self.game_over:
            renderer.draw_banner(surface, font, "GAME OVER - Press R to restart")
            # Final distance, centered just below the banner
            final_text = f"Final {distance_text}"
            text_w, _ = font.size(final_text)
            pos = ((surface.get_width() - text_w) // 2, surface.get_height() // 2 + 30)
            renderer.draw_text(surface, font, final_text, pos)
        else:
            renderer.draw_text(surface, font, distance_text, (10, 10))
```

What changed:

- **Tracking distance:** `self.distance` accumulates `SCROLL_SPEED` each frame, since the world scrolling past is what represents the helicopter traveling forward. It's stored in pixels and converted to meters with the new `PIXELS_PER_METER` constant (10), so you can tune the pace of the score in one place.
- **Stops at game over:** `update` returns early once `game_over` is set, so the counter freezes. I also only add distance on frames where no collision occurred, so the crash frame doesn't count.
- **During play:** `draw` shows `Distance: N m` in the top-left corner.
- **Game-over screen:** the top-left HUD is replaced by `Final Distance: N m`, centered just under the "GAME OVER" banner.
- **Restart:** `distance` is initialized in `_reset()`, so pressing R sets it back to zero along with everything else.

User (message 5)

Now Task 4. Add a shield mechanic. The player activates it with the SPACE key. While active, it protects the helicopter from exactly one obstacle collision. Show a clear visual indicator while it's active (for example, a colored circle around the helicopter, plus a "SHIELD" label on screen). The moment it absorbs a hit, the shield must disappear and the game must continue, and after that the helicopter is vulnerable again until the shield is activated once more. Make sure the helicopter doesn't die on the same collision that used up the shield, and that the shield isn't used up repeatedly on the same obstacle across several frames. Reset the shield when a new game starts. Tell me which files changed, and give me the complete updated version of each one.

Claude (reply 5)

Three files changed: `game/game_engine.py`, `game/obstacle.py`, and `game/renderer.py`. `main.py` and `helicopter.py` are unchanged, since `main.py` already forwards key presses to `engine.handle_keydown`.

game/game_engine.py

```
"""
GameEngine: owns the helicopter and all obstacles.
```

```
Collision detection and game over / restart are handled here (Task 2),
along with the distance score (Task 3) and the shield (Task 4).
"""

import random

import pygame

from game.helicopter import Helicopter
from game.obstacle import Obstacle
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 90
GAP_HEIGHT = 150
WALL_WIDTH = 60
SCROLL_SPEED = 3
PIXELS_PER_METER = 10   # scroll pixels that count as one meter of distance

COLOR_SHIELD_TEXT = (0, 120, 200)


class GameEngine:
    def __init__(self):
        self._reset()

    def _reset(self):
        """Put the game back into its starting state."""
        self.helicopter = Helicopter(x=100, y=HEIGHT / 2)
        self.obstacles = []
        self.frames_until_spawn = 0
        self.game_over = False
        self.distance = 0.0   # in scroll pixels; converted to meters for display
        self.shield_active = False

    def _distance_meters(self):
        return int(self.distance / PIXELS_PER_METER)

    def _spawn_obstacle(self):
        margin = 60
        gap_y = random.randint(margin + GAP_HEIGHT // 2, HEIGHT - margin - GAP_HEIGHT // 2)
        self.obstacles.append(Obstacle(
            x=WIDTH, gap_y=gap_y, gap_height=GAP_HEIGHT,
            wall_width=WALL_WIDTH, screen_height=HEIGHT, speed=SCROLL_SPEED,
        ))

    def _find_collision(self):
        """Return the first obstacle the helicopter is touching, or None.

        Obstacles whose hit was already absorbed by the shield are skipped,
        so the helicopter can safely finish passing through them.
        """
        heli_rect = self.helicopter.get_rect()
        for obstacle in self.obstacles:
            if obstacle.shield_hit:
                continue
            # Check the two wall rects separately; the gap between them
            # is not part of either rect, so flying through it is safe.
            if heli_rect.colliderect(obstacle.get_top_rect()):
                return obstacle
            if heli_rect.colliderect(obstacle.get_bottom_rect()):
                return obstacle
        return None

    def handle_input(self, keys_pressed):
        if self.game_over:
            return
        self.helicopter.handle_input(keys_pressed)

    def handle_keydown(self, key):
        if self.game_over:
            if key == pygame.K_r:
                self._reset()
        elif key == pygame.K_SPACE:
            self.shield_active = True
```

```
    def update(self):
        if self.game_over:
            return

        self.helicopter.update(HEIGHT)

        self.frames_until_spawn -= 1
        if self.frames_until_spawn <= 0:
            self._spawn_obstacle()
            self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

        for obstacle in self.obstacles:
            obstacle.update()
        self.obstacles = [o for o in self.obstacles if not o.is_off_screen()]

        hit = self._find_collision()
        if hit is not None:
            if self.shield_active:
                # Shield absorbs this one obstacle and is used up. Marking the
                # obstacle means it can't hurt us (or drain a future shield)
                # on the following frames while we're still overlapping it.
                self.shield_active = False
                hit.shield_hit = True
            else:
                self.game_over = True

        if not self.game_over:
            # Only count distance for frames the helicopter survives
            self.distance += SCROLL_SPEED

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.helicopter, self.obstacles,
                            shield_active=self.shield_active)

        distance_text = f"Distance: {self._distance_meters()} m"
        if self.game_over:
            renderer.draw_banner(surface, font, "GAME OVER - Press R to restart")
            # Final distance, centered just below the banner
            final_text = f"Final {distance_text}"
            text_w, _ = font.size(final_text)
            pos = ((surface.get_width() - text_w) // 2, surface.get_height() // 2 + 30)
            renderer.draw_text(surface, font, final_text, pos)
        else:
            renderer.draw_text(surface, font, distance_text, (10, 10))
            if self.shield_active:
                renderer.draw_text(surface, font, "SHIELD", (10, 40), COLOR_SHIELD_TEXT)
```

game/obstacle.py

```
"""
Obstacle: a scrolling wall pair with a gap the helicopter must fly
through.
"""

import pygame


class Obstacle:
    def __init__(self, x, gap_y, gap_height, wall_width, screen_height, speed):
        self.x = x
        self.gap_y = gap_y
        self.gap_height = gap_height
        self.wall_width = wall_width
        self.screen_height = screen_height
        self.speed = speed
        self.scored = False       # used for distance/pass tracking later
        self.shield_hit = False   # True once the shield has absorbed a hit from this obstacle

    def update(self):
        self.x -= self.speed

    def is_off_screen(self):
        return self.x + self.wall_width < 0
```

```
    def get_top_rect(self):
        top_height = self.gap_y - self.gap_height / 2
        return pygame.Rect(int(self.x), 0, self.wall_width, int(top_height))

    def get_bottom_rect(self):
        bottom_y = self.gap_y + self.gap_height / 2
        return pygame.Rect(int(self.x), int(bottom_y), self.wall_width, int(self.screen_height - bottom_y))
```

game/renderer.py

```
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (140, 200, 230)
COLOR_HELI = (60, 60, 70)
COLOR_OBSTACLE = (70, 150, 80)
COLOR_TEXT = (20, 20, 20)
COLOR_SHIELD = (0, 120, 255)

SHIELD_PADDING = 10   # how far the shield circle extends past the helicopter body


def draw_scene(surface, helicopter, obstacles, shield_active=False):
    surface.fill(COLOR_BG)
    for obstacle in obstacles:
        pygame.draw.rect(surface, COLOR_OBSTACLE, obstacle.get_top_rect())
        pygame.draw.rect(surface, COLOR_OBSTACLE, obstacle.get_bottom_rect())
    heli_rect = helicopter.get_rect()
    pygame.draw.rect(surface, COLOR_HELI, heli_rect, border_radius=4)
    if shield_active:
        radius = max(heli_rect.width, heli_rect.height) // 2 + SHIELD_PADDING
        pygame.draw.circle(surface, COLOR_SHIELD, heli_rect.center, radius, 3)


def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)


def draw_banner(surface, font, text):
    surf = font.render(text, True, (180, 40, 40))
    rect = surf.get_rect(center=(surface.get_width() // 2, surface.get_height() // 2))
    surface.blit(surf, rect)
```

What changed:

- **Activation:** pressing SPACE during play sets `shield_active = True`. Pressing it while the shield is already on does nothing, and SPACE is ignored on the game-over screen.
- **Indicator:** while active, the renderer draws a blue circle around the helicopter, and the HUD shows a "SHIELD" label under the distance counter. `draw_scene` gets a `shield_active` parameter that defaults to `False`, so any other callers still work.
- **Absorbing a hit:** `_find_collision` now returns the obstacle that was hit instead of a bool. If the shield is on, `update` turns it off and the game keeps running. Only a hit with no shield sets `game_over`, so the helicopter can't die on the same collision that used up the shield.
- **No repeat drain:** the helicopter usually overlaps a wall for several frames. When the shield absorbs a hit, that obstacle is flagged with the new `shield_hit` attribute and skipped by later collision checks. This is why `obstacle.py` changed. It also means that re-activating the shield while still overlapping that obstacle won't burn it on the same wall.
- **Vulnerable again:** once the shield is gone, the next *different* obstacle you touch ends the game, unless you press SPACE again first.

- **Restart:** `_reset()` sets `shield_active = False`, and the new game creates fresh obstacles with `shield_hit = False`.

One design choice to be aware of: there's no limit or cooldown on shield activations, so the player can re-arm it as often as they like. If your lab spec wants a limited supply or a cooldown, that's a small addition. Let me know.